códigofacilito

Bootcamp · Módulo 4: Integración App + DB

Base de datos con Supabase

Clase 1 de 6 · Supabase + Next.js



Rodrigo Méndez
rodrigomendez.dev

códigofacilito

¡Hola! 👋

Rodrigo Méndez, ingeniero de software full stack: React, React Native, Next.js y Node.js.

12 años construyendo software para empresas grandes.

- Construí desde cero la app que usan los repartidores de una cadena nacional de tiendas para el hogar.
- La app de una cadena de tiendas departamentales, con más de un millón de descargas.
- Fundé Piclink, una plataforma para fotógrafos que construí y opero solo.



rodrigomendez.dev
github.com/rodrigo2392

códigofacilito

Tabla de contenido

/01 El problema y el proyecto

/02 Supabase y la arquitectura

/03 Tablas, conexión y lectura

/04 Escribir datos

/05 Para cerrar la clase



códigofacilito

/01 El problema y el proyecto

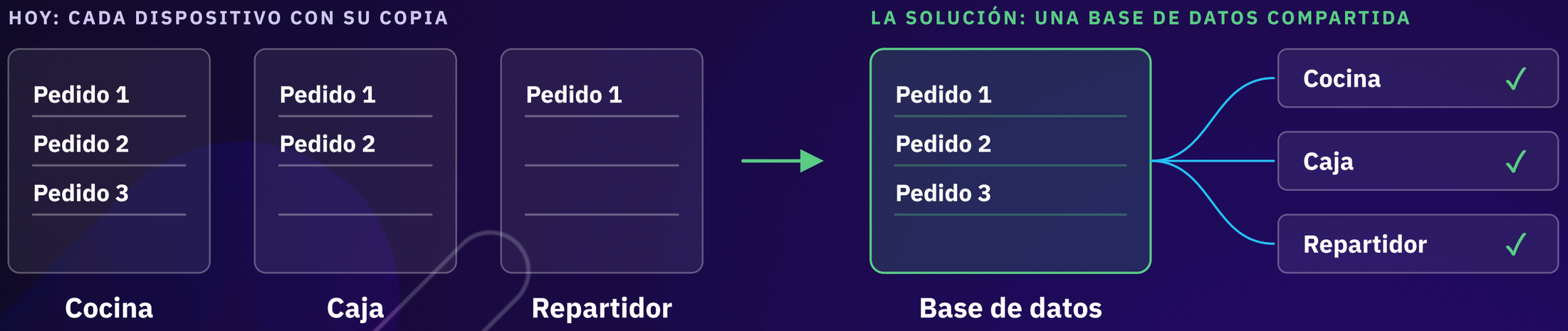
Qué vamos a construir, y por qué.



códigofacilito

EL PROBLEMA

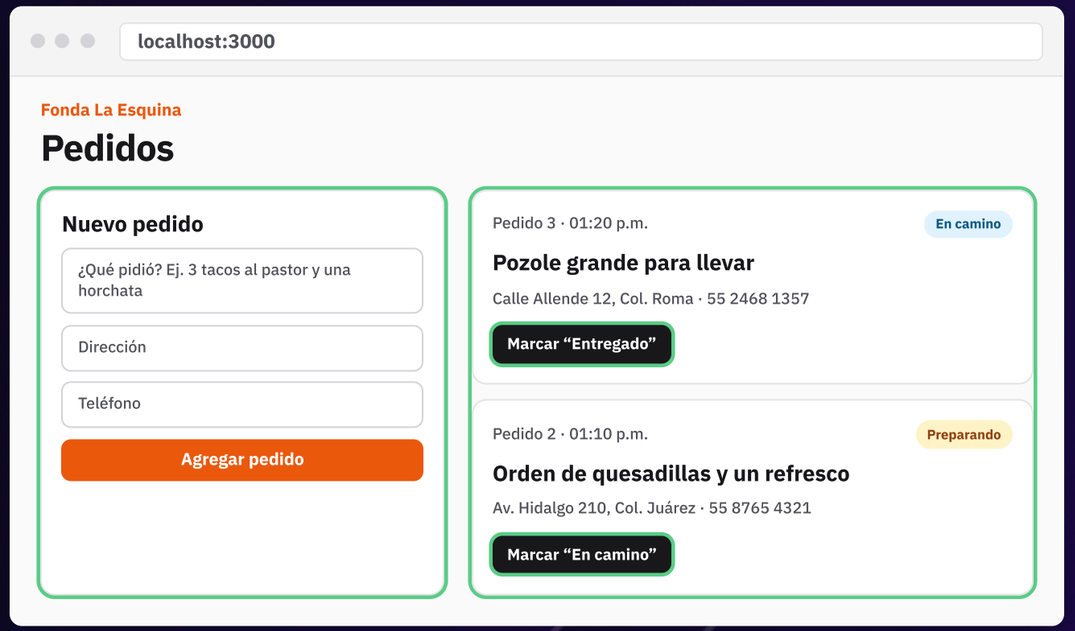
La cocina, la caja y el repartidor trabajan en dispositivos distintos. ¿Cómo ven los mismos pedidos, al mismo tiempo?



códigofacilito

LA META DE HOY

Tu app, con los pedidos en Supabase



1. **La lista viene de la base**
   La página consulta los pedidos en Supabase antes de renderizarse.
2. **El formulario guarda**
   Lo que escribes llega a la tabla, y lo ve cualquiera que abra la app.
3. **Los botones cambian el estado**
   Recibido, preparando, en camino y entregado.

RASTRO, SIGUE TU PEDIDO · UNA SOLA APP QUE CRECE CLASE A CLASE

Hoy					
CLASE 1	CLASE 2	CLASE 3	CLASE 4	CLASE 5	CLASE 6
Base de datos	Auth y RLS	Realtime	Edge Functions	Storage	Eventos y Broadcast

códigofacilito

/02

Supabase y la arquitectura

Qué es Supabase y dónde corre cada parte de la app.



códigofacilito

QUÉ ES SUPABASE

Postgres administrado, con Auth, Storage y Realtime

Base de datos

Postgres, con una API lista para usarse.

HOY

Auth

Usuarios, sesiones y permisos.

CLASE 2

Realtime

Los cambios llegan al navegador al instante.

CLASE 3

Edge Functions

Tu código, corriendo en Supabase.

CLASE 4

Storage

Archivos con permisos.

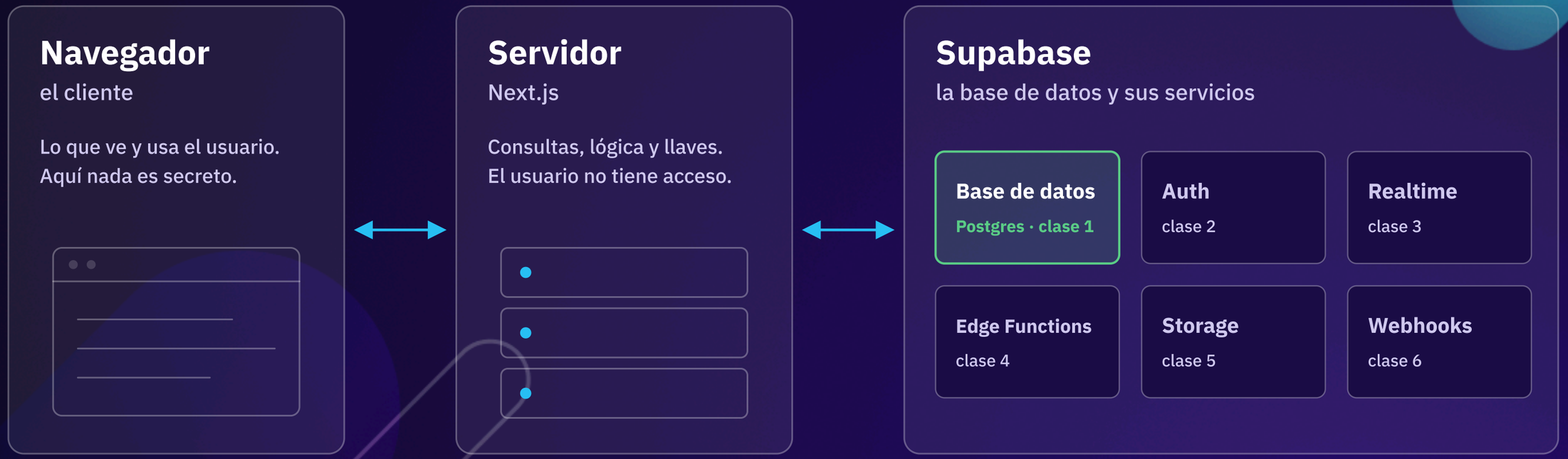
CLASE 5

> Es Postgres: SQL estándar. Todo se administra desde el panel, sin instalar nada.

códigofacilito

LA ARQUITECTURA DEL CURSO

Navegador, servidor y Supabase

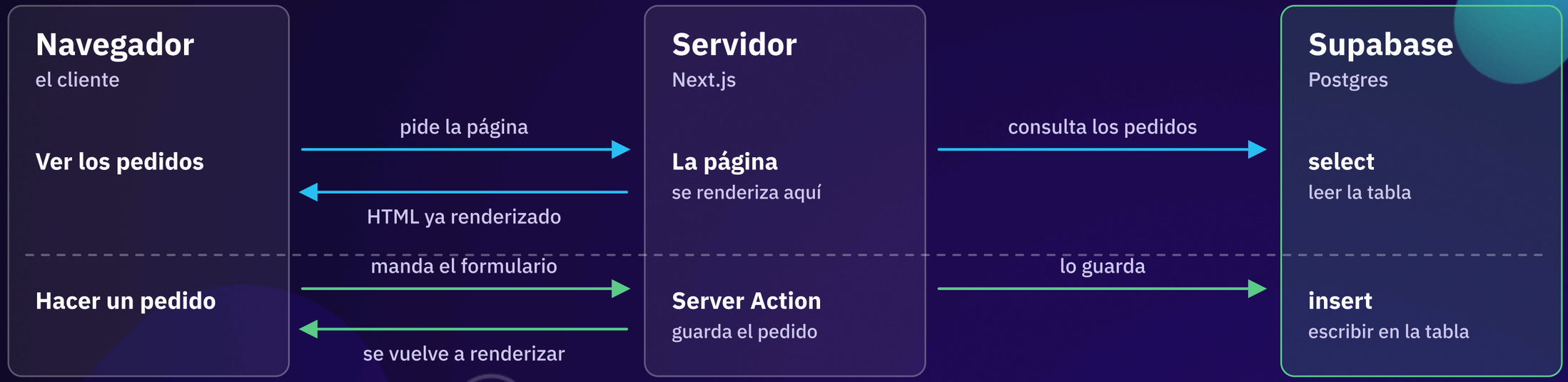


> Cada clase suma un servicio de Supabase. Hoy: la base de datos.

códigofacilito

IDEA 1 DE 3 · LO ÚNICO DE NEXT.JS QUE USAMOS HOY

Se lee y se escribe desde el servidor



`async function Inicio()`

La página consulta los datos antes de renderizarse: **Server Component**.

`"use server"`

El formulario llama a una función que corre en el servidor: **Server Action**.

códigofacilito

/03

Tablas, conexión y lectura

Qué es una tabla, cómo se conecta tu app y cómo se lee.



códigofacilito

IDEA 2 DE 3

Una tabla valida lo que guarda

pedidos

id bigint · identity	created_at timestamptz · now()	descripcion text · not null	direccion text · not null	telefono text · not null	estado text · default 'recibido'
1	14 oct 13:05	3 tacos al pastor y una horchata	Calle Morelos 45	55 1234 5678	recibido
2	14 oct 13:10	Orden de quesadillas	Av. Hidalgo 210	55 8765 4321	preparando
3	14 oct 13:20	Pozole grande para llevar	Calle Allende 12	55 2468 1357	en_camino
~~4~~	~~14 oct 13:30~~	*vacío*	~~Calle Juárez 8~~	~~55 1111 2222~~	~~recibido~~

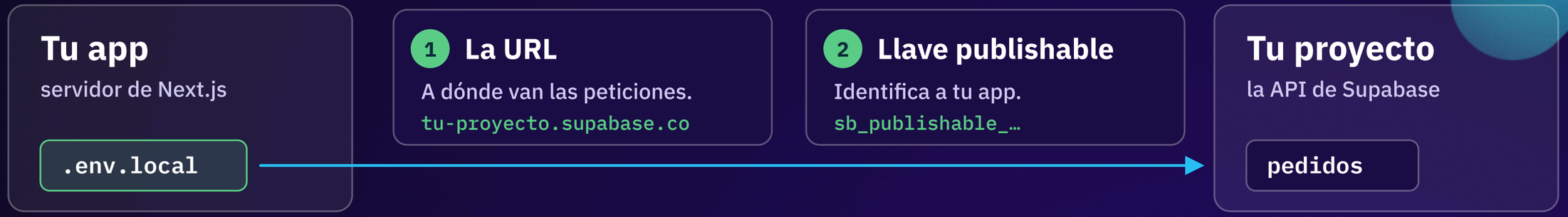
✕ **Rechazado: descripcion es not null. La base valida cada fila antes de guardarla.**

Fila un pedido **Columna** un campo, con tipo y restricciones **id** la llave primaria: identifica cada pedido

códigofacilito

CONECTAR NEXT.JS CON SUPABASE · IDEA 3 DE 3

La llave publishable es pública



Los dos van en .env.local: variables de entorno, fuera del código y del repo.

LLAVE PUBLISHABLE · `sb_publishable_`

Pública por diseño

Hecha para ir en el navegador: cualquiera la puede ver. Solo puede lo que la base le permita.

LLAVE SECRETA · `sb_secret_`

Acceso total

Se salta RLS. Solo en el servidor; nunca en el navegador ni en el repo. Clase 4.

> Lo que protege los datos son las políticas RLS de la base, no esconder la llave. Eso es la clase 2.

códigofacilito

LEER DESDE EL SERVIDOR

Leer: una consulta con select

¿DE QUÉ TABLA?
pedidos `.from("pedidos")`

¿QUÉ COLUMNAS?
todas `.select("*")`

¿EN QUÉ ORDEN?
más nuevos primero `.order("created_at", { ascending: false })`

EL RESULTADO: UN ARREGLO

3	Pozole grande para llevar	en camino
2	Orden de quesadillas	preparando
1	3 tacos al pastor y una horchata	recibido

> La consulta corre en el servidor, dentro de la página.
> En SQL: `select * from pedidos order by created_at desc`

códigofacilito

/04 Escribir datos

insert y update desde una Server Action.



códigofacilito

CRUD: LAS CUATRO OPERACIONES

Leer, guardar, cambiar y borrar

Leer

`select`

Traer filas: la lista.

HOY

Guardar

`insert`

Agregar una fila: el formulario.

HOY

Cambiar

`update`

Cambiar datos de una fila: los botones.

HOY

Borrar

`delete`

Quitar filas.

MÁS ADELANTE

> Guardar y cambiar corren en el servidor, en una Server Action. Después, `revalidatePath` le indica a Next.js que vuelva a renderizar la página.

códigofacilito

EL ERROR DEL DÍA

Guardas un pedido y no aparece

1

Guardas

La Server Action hace el `insert` y no marca error.

2

No aparece

La página sigue igual: Next.js no la volvió a renderizar.

3

Sí se guardó

En el panel ya está, y al recargar aparece.

4

revalidatePath

Le indica a Next.js que vuelva a renderizar la ruta.

> Después de escribir en la base desde una Server Action, llama a `revalidatePath` con la ruta que muestra el dato.

códigofacilito

CAMBIAR EL ESTADO

Cada botón cambia un solo pedido

Recibido → Preparando → En camino → Entregado

pedidos

id	descripcion	estado
1	3 tacos al pastor y una horchata	recibido
2	Orden de quesadillas	~~preparando~~ **en_camino**
3	Pozole grande para llevar	en_camino

1 **Qué cambia**

`.update({ estado: "en_camino" })`

2 **En cuál: el filtro**

`.eq("id", 2)` solo toca el pedido 2.

✕ **En SQL, un update sin filtro cambia todas las filas de la tabla.**

códigofacilito

/05

Para cerrar la clase

Lo que te llevas, la tarea y lo que sigue.



códigofacilito

LO QUE TE LLEVAS HOY

Tres ideas

1

Se lee y se escribe desde el servidor

La página consulta en el servidor y el formulario guarda con una Server Action.

2

Una tabla valida lo que guarda

Cada columna tiene tipo y restricciones; la base rechaza lo que no cumple.

3

La llave publishable es pública

Cualquiera la puede ver. Lo que protege los datos son las políticas RLS.

códigofacilito

TAREA · PARA EL VIERNES

Construye tú solo lo que viste hoy

CÓMO SÉ QUE YA QUEDÓ

- [ ] Tu proyecto de Supabase tiene su tabla de pedidos.
- [ ] La página lee los pedidos de esa tabla.
- [ ] El formulario guarda un pedido nuevo y aparece en la lista.
- [ ] Cada botón mueve su pedido al estado que sigue.

¿NO TE SALIÓ?

No pasa nada

Toma la base de la clase 2: ya trae todo esto hecho. Compárala con lo tuyo y encuentra qué te faltó.

códigofacilito

Recursos para profundizar

- Supabase con Next.js, paso a paso: supabase.com/docs/guides/getting-started/quickstarts/nextjs
- Tablas en Supabase: supabase.com/docs/guides/database/tables
- Leer datos en Next.js: nextjs.org/docs/app/getting-started/fetching-data
- Guardar datos con Server Actions: nextjs.org/docs/app/getting-started/mutating-data



códigofacilito

PROBLEMA DE SEGURIDAD

Hoy tu tabla está expuesta



- ✕ Leer todos los pedidos, con direcciones y teléfonos
- ✕ Crear pedidos falsos
- ✕ Cambiar el estado de cualquier pedido

Nunca así en producción. En la clase 2 lo cerramos con autenticación y políticas RLS.